

Multi-Agent Drone Capture and Evasion via Adversarial Reinforcement Learning

Rishikesh Narayana & Owen Murphy | Computer Systems Lab 2026 | Yilmaz & Gabor



Simulation Demo

Introduction

Reinforcement learning (RL) enables agents to learn decision-making by interacting with an environment and optimizing cumulative rewards (Figure 1). It has been widely applied in robotics and autonomous navigation, where agents must adapt to dynamic and uncertain conditions. In adversarial settings, competing agents drive adaptive and strategic behaviors in scenarios such as pursuit-evasion and target capture, requiring algorithms that learn robust policies despite non-stationary conditions.

Vision-based perception allows agents to interpret and react to their surroundings using stereo cameras (Figure 2), processed via AprilTag detection, optical flow, or feature extraction.

These techniques support object detection, localization, and relative positioning for navigation, target tracking (Figure 3), and obstacle avoidance. Integrating visual inputs into the RL observation space enables real-time informed decision-making.

Policy-gradient methods like Proximal Policy Optimization are widely used for continuous control tasks due to their stability and efficiency. Domain randomization — varying lighting, camera noise, and scene textures during training — improves robustness and generalization. Together, these approaches enable autonomous systems that are adaptive, perceptive, and resilient in complex competitive environments.

Methodology

A vision-based pipeline uses OpenCV and AprilTag detection to extract bearing and distance from the drone's camera feed with under 5% error. These outputs and state variables (position, velocity) form the observation space, validated through logged test flights. The agent uses a discrete action space (forward thrust, yaw left/right, hover) at 0.1 s intervals.

Training uses PPO via Stable-Baselines3 over 20,000+ episodes, logging reward, win rate, and crash rate. Three variants are evaluated: Vanilla PPO (unstable), KL-PPO (plateaus early), and Adaptive Clip PPO (AC-PPO), which best balances stability and performance.

Domain randomization varies lighting ($0.5\times-2\times$), physics ($\pm 10\%$), and sensor noise for generalization. Policies are evaluated against scripted, random, and self-play opponents using win rate, dwell time (>0.2 s), and crash rate ($<10\%$).

For sim-to-real transfer, trained PyTorch models are exported and optimized with TensorRT, with hardware setup including firmware configuration, PID tuning, and stable manual hover, enabling real-world deployment on a 5-inch drone with a Jetson Nano for real-time inference.

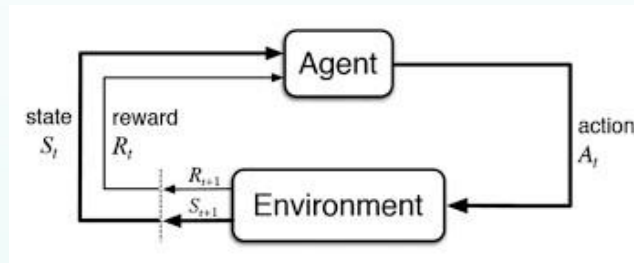


Figure 1. Single Agent reinforcement learning basic pipeline

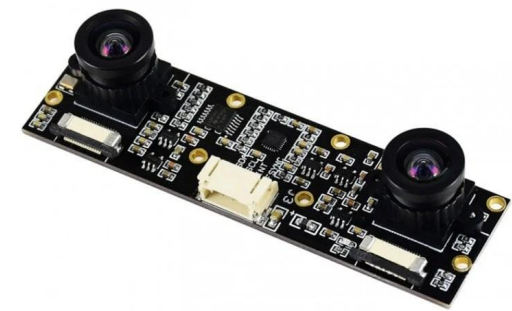


Figure 2. IMX219-83 dual-module Stereo camera for depth perception

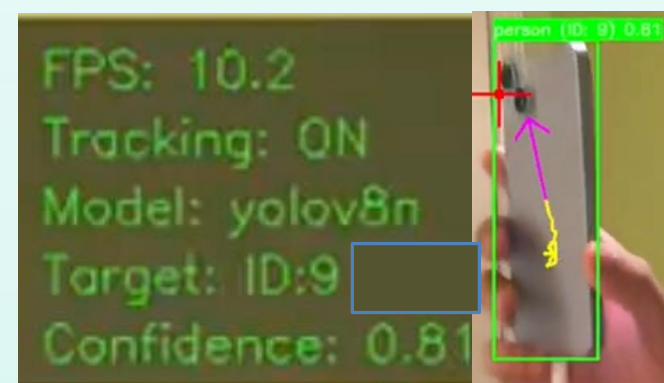


Figure 3. yoloV8 tracking velocity and position relative to center frame.

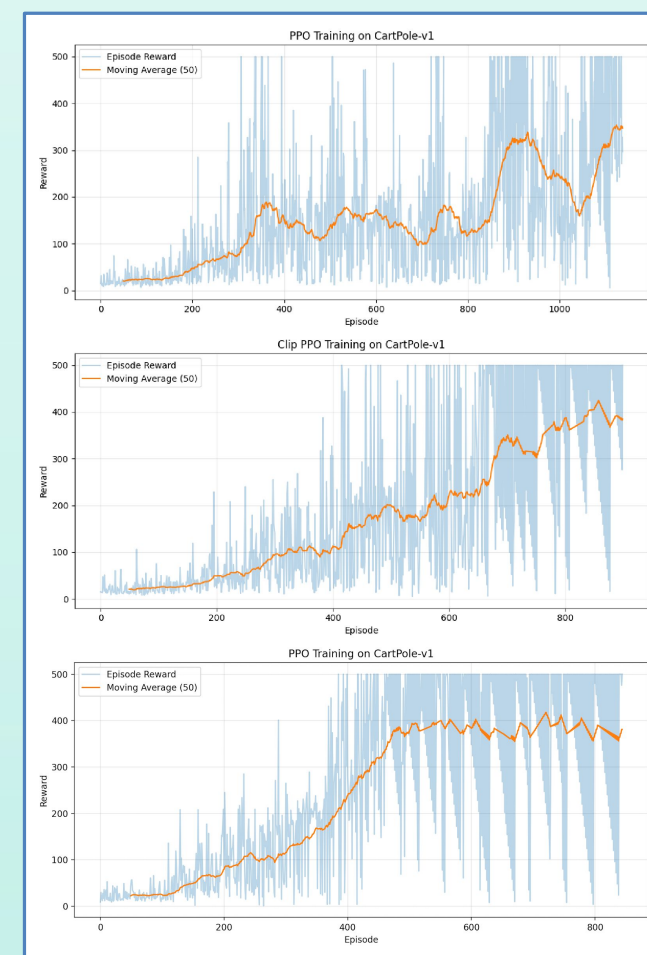


Figure 4. Vanilla PPO (Top), Adaptive Clip (Middle), KL-PPO (Bottom) reward graphs over 500 CartPole-v1 episodes

Hardware

Autonomous drone testing initially used 5-inch drones with Kakute H7 flight controllers and Jetson Nano computers for real-time inference, with PX4/Betaflight firmware, PID tuning, and calibrated sensors. Due to technical failures and difficulty implementing low-level Python control, the project transitioned to DJI Tello EDU drones, which offer a robust prebuilt Python SDK that significantly reduced hardware integration overhead.

The Tello EDU lacks onboard GPS, making absolute positioning unreliable; IMU dead-reckoning provides mediocre position estimates, with accumulated drift necessitating reliance on AprilTag-based visual localization. Flight logs and episode data are written to JSON files in real time, supporting post-hoc analysis and policy evaluation across hardware trials.



Figure 5. Carbon Fiber drone frame and motor installation.



Figure 6. DJI Tello EDU drone out-of-the-box

Simulation

Built a custom PyBullet environment for 2-drone adversarial training in a bounded 10 m arena. Drones have independent simulated cameras, continuous $[128\times 128]$ observation spaces, and dt jitter to simulate real-world latency. Scripts handle alternating PPO updates every 512 steps with the opponent model frozen.

The environment incorporates domain randomization (lighting, camera noise, URDF textures), reward shaping (hotspot dwell time, constant velocity penalties, dodge bonuses), and curriculum learning via progressively closer starting positions. Checkpoint saves and JSON progress files enable long simulations to resume without data loss.

The codebase uses modular Python scripts covering environment setup (URDF spawning, reset, boundary planes), perception (AprilTag detection, observation logging), RL integration (SB3 PPO, stepSimulation loops), and training utilities. All modules communicate via PyBullet's Python API, enabling seamless sim-to-real transfer to physical Tello deployment.

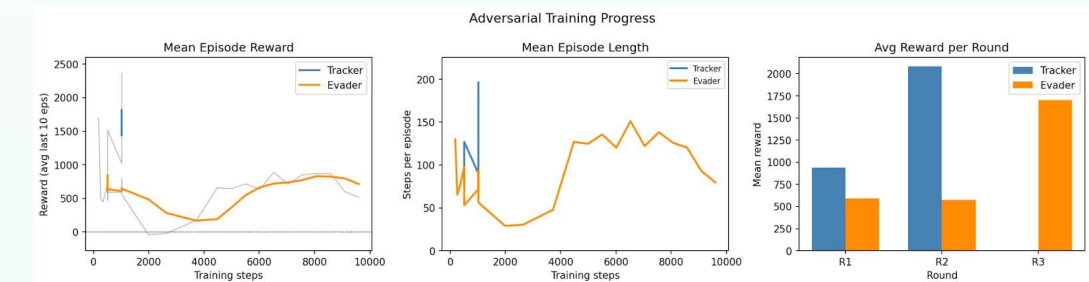


Figure 7. Training rewards for evader and tracker by episode

Simulation Results

The simulation experiments in the PyBullet environment demonstrated that the PPO-based RL agent effectively learned to navigate the arena and interact with hotspots. Training over 20,000 episodes resulted in a final average cumulative reward of ~ 185 per episode, with the learning curve showing steady improvement after 8,000 episodes. Hotspot dwell time increased from 0.05 s initially to 0.28 s on average, while the crash rate decreased from $\sim 18\%$ to 7%, indicating that the agent learned to balance exploration with safety.

Domain randomization across lighting ($0.5\times-2\times$ intensity, varying angles) and physics parameters ($\pm 10\%$ thrust, wind variation) produced a robust policy capable of handling environment variability without significant performance loss. Observation logging verified that the 12-dimensional observation vector accurately captured positional, velocity, and target-related states necessary for informed decision-making.

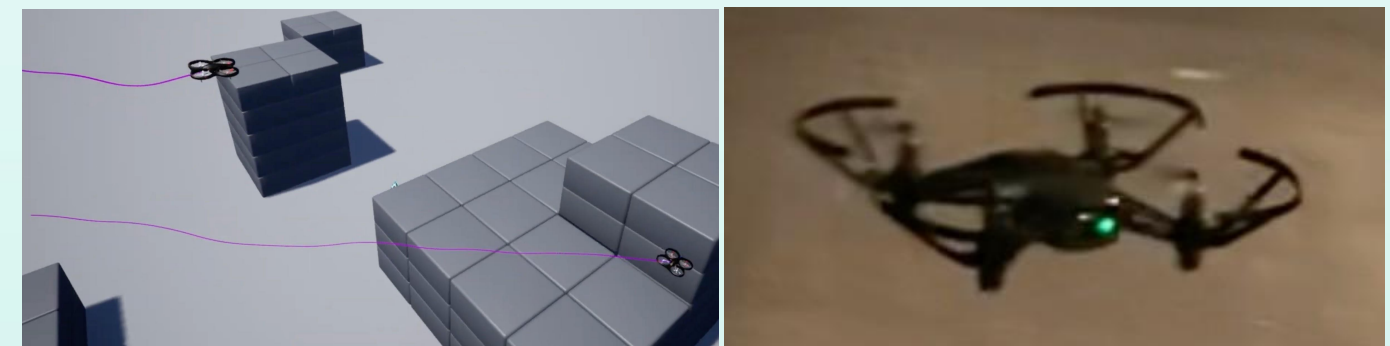


Figure 8. Tracking and evading drones traces Figure 9; DJI Tello Tracking April Tag in "Blocks" environment

Real-World Results

Real-world deployment trials demonstrated successful autonomous pursuit behavior using two DJI Tello EDU drones in a controlled indoor environment. The pursuer drone utilized a YOLO-based AprilTag detection pipeline to identify and localize the evader drone, which carried a mounted AprilTag marker, in real time from its front-facing camera stream. The trained RL policy processed the detected tag's position and estimated relative distance to generate velocity commands, enabling the pursuer to actively track the evader as it moved through the arena. Across multiple trial runs, the pursuer successfully maintained a target standoff distance of approximately one to two feet from the evader, adjusting its heading and speed dynamically in response to evader movement.

These results validate a proof of concept for vision-based autonomous pursuit in a low-speed, obstacle-free, controlled indoor setting, confirming that the trained policy and detection pipeline can transfer from simulation to real hardware under favorable conditions, while highlighting areas for improvement including latency reduction, lighting robustness, and tighter distance regulation under more dynamic evader maneuvers.